

Resolución de Actividades: Arquitectura Front-End

Actividad 3 y 4

Actividad 3: Diseño de un template

Para esta actividad, se ha seleccionado el dominio de **Blog**. El diseño se centra en un portal de artículos sobre reglas y mecánicas de juegos de rol. A continuación, se detallan las decisiones arquitectónicas implementadas mediante Bootstrap 3:

1. Layout de doce columnas para tres rangos de ancho

Se aplicó el patrón responsivo sugerido para blogs, consistente en un contenido central y un sidebar lateral. La grilla se distribuyó de la siguiente manera:

- **Escritorio amplio (lg):** El contenido ocupa una proporción de 9/12 (`col-lg-9`) y el sidebar 3/12 (`col-lg-3`).
- **Tablet (sm):** Para optimizar la lectura, la proporción se ajusta a 8/12 (`col-sm-8`) para el contenido principal y 4/12 (`col-sm-4`) para el sidebar.
- **Móvil (xs):** Ambos contenedores utilizan el ancho completo (`col-xs-12`), produciendo un reflujo donde el sidebar se apila estructuralmente bajo los artículos.

2. Navegación y Componentes Principales

- **Navegación:** Se implementó una barra superior semántica (`<nav>`) que incluye un menú principal. En anchos reducidos, este componente se transforma en un menú colapsable (patrón de hamburguesa) para preservar el espacio útil.
- **Componentes:** Se utilizaron paneles (`panel-default`) para estructurar tanto los artículos principales (empleando la etiqueta `<article>`) como la lista de categorías del sidebar (`list-group`).

3. Estados de la interfaz

Se incorporaron contenedores ocultos que permiten gestionar la experiencia del usuario de manera controlada frente a distintos escenarios operacionales:

- **Estado de carga:** Un indicador visual diseñado para mostrarse mientras se solicitan los datos al backend.
- **Estado vacío:** Un mensaje informativo (`alert-info`) que se activa si una categoría no contiene artículos.
- **Estado de error:** Un mensaje de advertencia (`alert-warning`) preparado para una eventual falla en la comunicación o timeout.

4. Reglas de accesibilidad

El template no solo atiende a la geometría, sino también a la semántica y accesibilidad funcional:

- Uso estricto de HTML5 semántico (`<nav>`, `<main>`, `<aside>`, `<article>`) en lugar de contenedores genéricos.
- Implementación de etiquetas `aria-label` para identificar regiones clave (“Navegación principal”, “Barra lateral”) para lectores de pantalla.
- Atributo `aria-live="polite"` en el contenedor principal, para notificar cambios de estado dinámicos (carga, error) de forma no intrusiva.
- Clase `sr-only` en el botón de navegación móvil para asegurar su lectura por tecnologías de asistencia.

5. Evidencia de pruebas en cuatro anchos diferentes

Se validó la adaptabilidad del layout sometiendo la interfaz a distintas resoluciones, verificando que la jerarquía y legibilidad se conservaran en todos los escenarios.

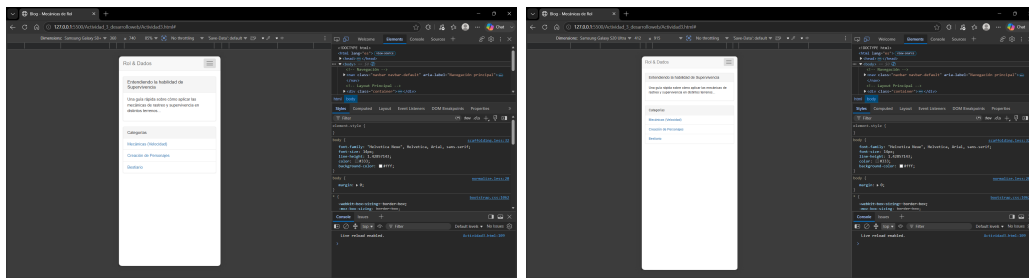


Figura 1: Prueba Móvil: Colapso del menú de navegación y reflujo del sidebar.

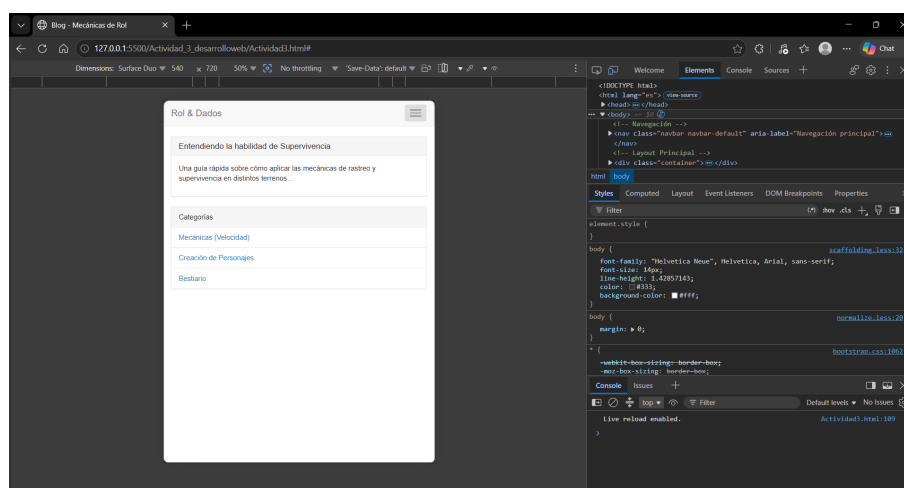


Figura 2: Prueba Tablet / Intermedia: Transición del layout a proporción 8/12 y 4/12.

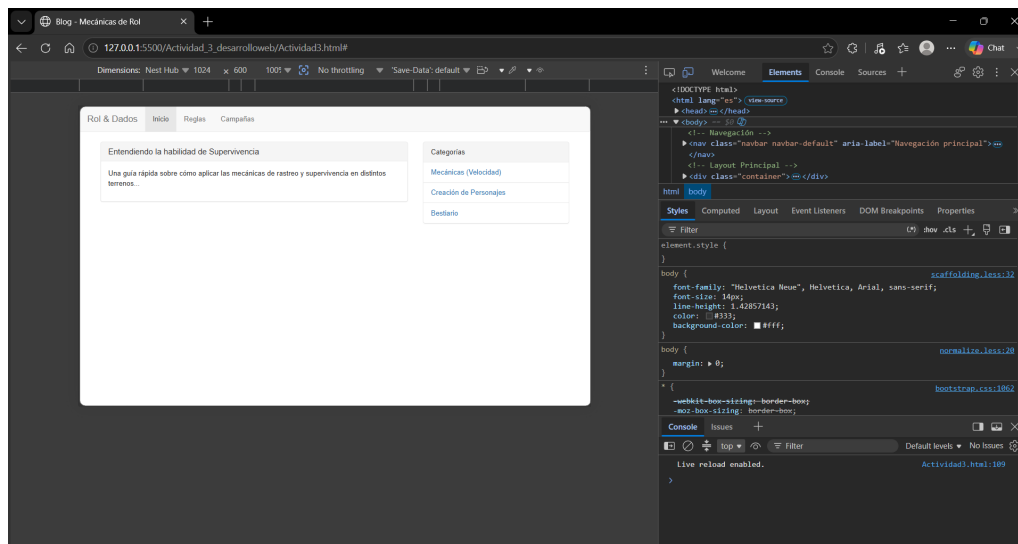


Figura 3: Prueba Escritorio: Expansión del menú y distribución horizontal (9/12 y 3/12).

Actividad 4: Prueba de resiliencia

El Concern (El Problema)

En el contexto del portal de artículos de rol, existe el riesgo de que la API encargada de proveer los datos (los “pergaminos” o entradas del blog) experimente una alta latencia (simulada en 10 segundos) o falle completamente al intentar conectar con el servidor. Si esto ocurre sin un manejo adecuado, el usuario se enfrentaría a una pantalla en blanco o a una interfaz rota, provocando confusión y una mala experiencia de navegación.

La Decisión

Para mitigar este problema, se implementó una estrategia de *degradación elegante* mediante JavaScript. Se decidió estructurar la interfaz de modo que la falla en la carga de los artículos no afecte la disponibilidad de los demás componentes. Mientras se esperan los datos, se muestra un estado de carga explícito. Si el tiempo se agota (timeout), se oculta el contenedor de los artículos y se despliega un mensaje de advertencia (**alert-warning**), manteniendo intactos y completamente funcionales tanto la barra de navegación superior como el sidebar lateral de categorías. Esto asegura que el usuario no pierda el contexto de la aplicación y pueda seguir interactuando con otras secciones.

La Evidencia

Se simuló el timeout de 10 segundos utilizando la función `setTimeout`. A continuación, se presenta la evidencia de la interfaz manteniendo su funcionalidad y estructura principal a pesar del retraso y eventual error de carga en múltiples resoluciones.

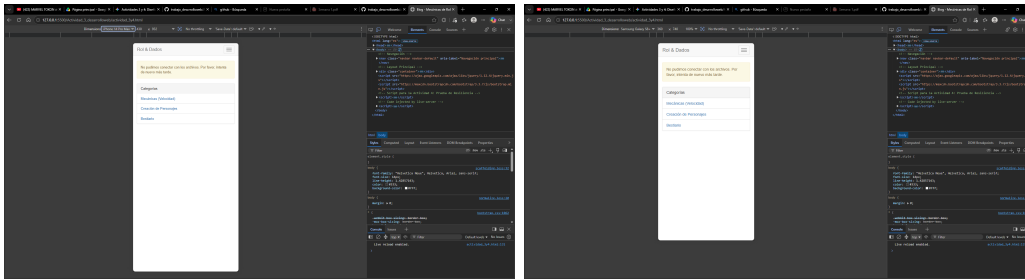


Figura 4: Evidencia de degradación elegante en dispositivos móviles. El menú hamburguesa y las categorías se mantienen funcionales frente al error de conexión.

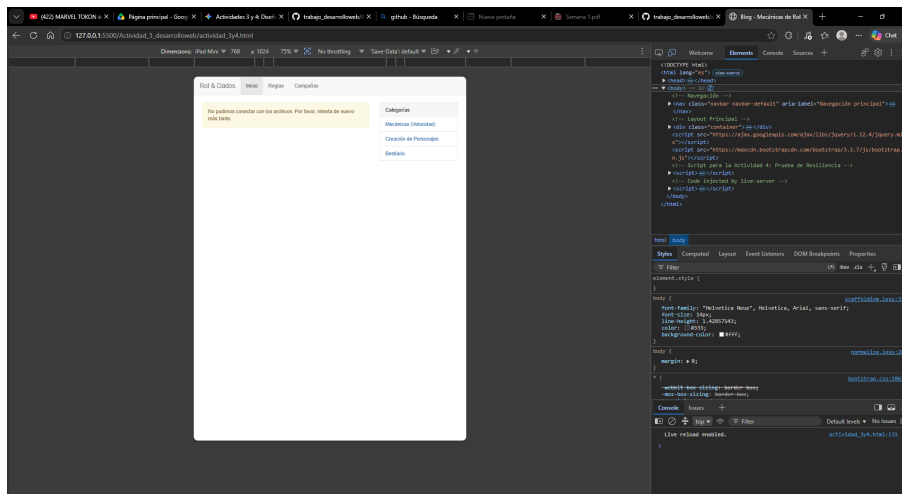


Figura 5: Evidencia de degradación elegante en tablet (iPad Mini). La estructura de columnas se adapta preservando la barra lateral intacta.

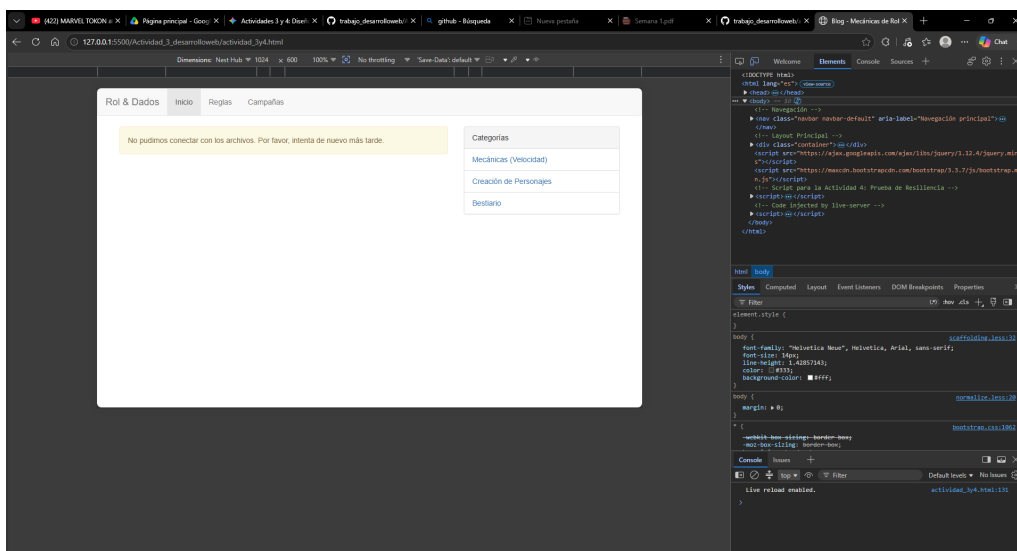


Figura 6: Evidencia de degradación elegante en pantalla amplia (Nest Hub). Distribución de layout principal conservada a pesar del timeout de la API.